EE485 Spring 2024 – PROJECT FINAL REPORT

Classification of Galaxies Based On Machine Learning

Yigit Turkmen

22102518

25.12.2024

Introduction

Galaxies are fundamental components of the universe, classified mainly as spiral or elliptical. Spiral galaxies have distinct arms and active star formation, while elliptical galaxies are more spherical and composed of older stars. Classifying galaxies helps astronomers understand their formation and evolution.

With the vast amount of galaxy data available from surveys like Sloan Digital Sky Survey, manual classification is impractical. Machine learning offers a solution to automate this process, making it faster and reducing human bias.

Problem Description

This project aims to develop a binary classification system to distinguish between spiral and elliptical galaxies using features from the Galaxy Zoo dataset. I will implement and evaluate K-Nearest Neighbors (k-NN), Gradient Boosting, and a Neural Network to determine the best approach for classification. By comparing these methods, I aim to achieve high accuracy and identify key features that differentiate galaxy types.

Dataset Description

The dataset used in this project is derived from the **Galaxy Zoo (https://data.galaxyzoo.org/)** project, which contains labeled data from crowdsourced galaxy classifications. The dataset includes features such as
NVOTE: Number of votes received for classification.
P_CW: Probability of being a clockwise spiral galaxy.
P_ACW: Probability of being an anticlockwise spiral galaxy.
P_EDGE: Probability of being an edge-on galaxy.
P_DK: Probability of being a galaxy with unknown type (don't know).
P_MG: Probability of being a merger.
These features are used to distinguish between spiral and elliptical galaxies. The data has been pre-processed to remove missing values and ensure consistency, making it suitable for training machine learning mode. It has 660k samples and %70 of samples are labeled as UNCERTAIN which are removed for my purpose. Remaining samples had a 1/3 ratio of ELLIPTICAL VS SPIRAL and this ratio was handled to be %47 vs %53 to make the distribution even.

Simulation Setup

The Python language was used for training and evaluating the models. The Pandas library was used to load data from the CSV file, while NumPy was used for matrix operations and calculations. For visualization, Matplotlib were utilized to create plots and analyze data distributions. Additionally, from Sklearn library train-test-split method used to split the data.

1. Dataset Preprocessing

Preprocessing is a crucial step that should be applied to data to enhance the accuracy of interpretations and predictions.

a) Standardization: Since features for my report is already in a standartized manner. I did not implemented on the dataset. However in case of anything, I wrote the code for it from scratch for possible use case with the following formula:

$$X_{j\ standardized} = (X_j - \mu_j)/\sigma\_j$$

Where X_j: Original feature values (column j of the dataset).

μ_j : Mean of feature j, calculated as:

$$\mu_j = \frac{1}{n}\sum X_{ij}\ over\ all\ i$$

Where n is the number of samples, and Xi,j is the value of feature j for the i-th sample.

σ_j: Standard deviation of feature j, calculated as:

$$\sigma_j = (\frac{1}{n}\sum (X_{ij-}\ \mu_j)\text{^}2\ ) ** 0.5 \quad over\ all\ i$$

b) Addressing Data Imbalance: In the original dataset there was 660k samples and %70 of samples are labeled as UNCERTAIN which are removed to eliminate the unnecessary parts. Remaining samples had a 1/3 ratio of ELLIPTICAL VS SPIRAL and this ratio was handled to be %47 vs %53 to make the distribution even.

c) Feature Selection: In the original dataset there were also seven more features three of them are useless (giving no information about the characteristics of the galaxy):
OBJID: Object ID, a unique identifier for each galaxy.
RA: Right Ascension, a coordinate in the sky (similar to longitude).
DEC: Declination, another coordinate in the sky (similar to latitude)

However, there were also four more giving explicitly the output. Therefore, I removed them from my features to make a fair classification:
P_EL: Probability of being an elliptical galaxy.
P_CS: Probability of being a completely smooth galaxy.
P_EL_DEBIASED: Debiased probability of being an elliptical galaxy.
P_CS_DEBIASED: Debiased probability of being a completely smooth galaxy.

2. Dataset Analysis

Feature Ranking Using Correlation Matrix: I prioritized the features based on their correlations with one another and with the target label. The correlation between features was assessed using the corrolation matrix.

Correlation Between Features and Labels:

The correlation coefficients were determined using

$$R_j = \frac{Cov(x_j\ ,\ y)}{\sqrt{Var(x_j)\cdot Var(y)}} = \frac{\sum_{i=0}^{n}(x_{ij} - \overline{x_j})(y_i - \overline{y})}{\sqrt{\sum_{i=0}^{n}(x_{ij} - \overline{x_j})^2 \cdot \sum_{i=0}^{n}(y_i - \overline{y})^2}}$$

Correlation Matrix between originally all features and the labels (Table and the Plot):

Figure 1: Correlation matrix plot.

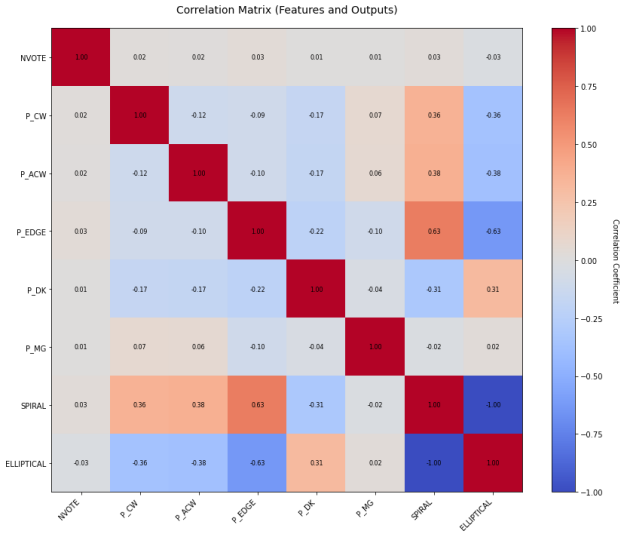


Table 1 and 2: Correlation matrix tables.

	NVOTE	P_CW	P_ACW	P_EDGE	P_DK	P_MG	SPIRAL	ELLIPTICAL
NVOTE	1.000000	0.017893	0.015509	0.034111	0.007773	0.006602	0.027301	-0.027301
P_CW	0.017893	1.000000	-0.115005	-0.094080	-0.166024	0.069527	0.361728	-0.361728
P_ACW	0.015509	-0.115005	1.000000	-0.096158	-0.174216	0.057499	0.377062	-0.377062
P_EDGE	0.034111	-0.094080	-0.096158	1.000000	-0.215527	-0.100507	0.626116	-0.626116
P_DK	0.007773	-0.166024	-0.174216	-0.215527	1.000000	-0.039116	-0.312895	0.312895
P_MG	0.006602	0.069527	0.057499	-0.100507	-0.039116	1.000000	-0.024592	0.024592
SPIRAL	0.027301	0.361728	0.377062	0.626116	-0.312895	-0.024592	1.000000	-1.000000
ELLIPTICAL	-0.027301	-0.361728	-0.377062	-0.626116	0.312895	0.024592	-1.000000	1.000000

	NVOTE	P_CW	P_ACW	P_EDGE	P_DK	P_MG
SPIRAL	0.027301	0.361728	0.377062	0.626116	-0.312895	-0.024592
ELLIPTICAL	-0.027301	-0.361728	-0.377062	-0.626116	0.312895	0.024592

From here, it can be seen that P_EDGE is the best feature with corr. 0.626 followed by P_ACW and P_CW where NVOTE and P_MG are the least corrolated features with the output label.

Implemented Algorithms

1) K-Nearest Neighbors (KNN)

K-Nearest Neighbors (KNN) is a fundamental and versatile algorithm often employed for classification tasks in machine learning. This simplicity and effectiveness make it an excellent choice for many applications, including our study. KNN is a supervised learning algorithm because it uses labeled data to train a model that predicts the label or output for unseen test data. Refer to [4]

The algorithm can be summarized as follows:

1. A test data instance is selected.
2. The distances between this test data and all training data points are computed. Since the data is multidimensional, this calculation requires vector operations.
3. The computed distances are then compared, and the smallest kkk distances are identified.
4. Using the indices of the kkk smallest distances, the corresponding outputs (labels) of the kkk nearest neighbors in the training data are selected.
5. The algorithm determines the probability of the test data instance belonging to a specific class (e.g., class 1) by calculating the proportion of nearest neighbors in that class. This probability is often referred to as γ.
6. A decision threshold is applied to the probability to assign a class label to the test instance. Alternatively, some implementations simply assign the label based on the majority class among the k nearest neighbors.

In my implementation, I did not adopted a probabilistic I simply choose the predicted label as the most common in the nearest neighboors, if there are k=3, it should have atleast 2 closest sample points to be chosen similary it need to be atleast 3 for k=5 and atleast 6 for k = 10.

Feature scaling is essential in KNN as it relies on distance metrics like Euclidean distance to measure similarity. Without scaling, features with larger ranges can dominate the calculation, leading to biased predictions. Normalization or standardization ensures all features contribute equally. Despite its simplicity and versatility, KNN has limitations: it is computationally expensive as it calculates distances for all training points, sensitive to noisy data, and struggles with imbalanced classes where the majority class can dominate predictions. Selecting the optimal kkk value is another challenge, often addressed using methods like k-fold cross-validation. However, KNN's non-parametric nature and ability to handle multi-class problems make it a valuable algorithm. See the figures below for visual representation:

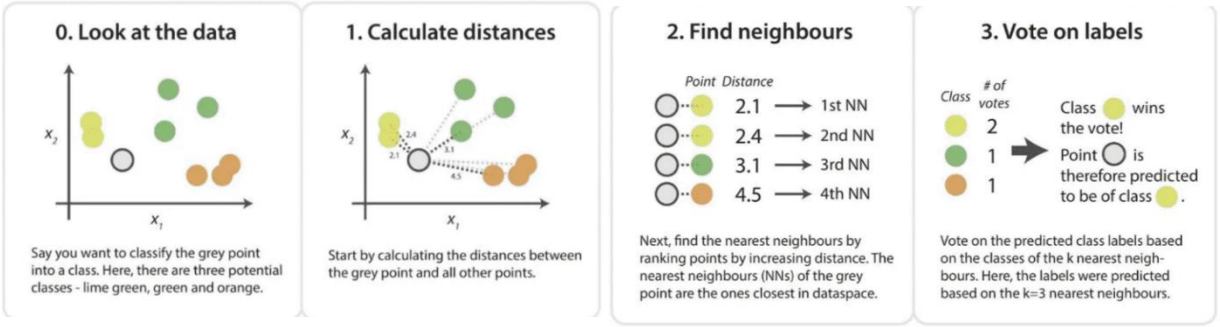


Figure 2: Visualization for KNN algorithm. [1]

2) Gradient Boosting

Gradient Boosting is an ensemble learning technique that excels at solving both classification and regression problems. It is particularly powerful due to its iterative approach, combining multiple weak learners (usually decision trees) to create a strong predictive model. In this project, I chose Gradient Boosting because of its ability to capture complex relationships in the data, which simple algorithms often miss also it is super good in tabular data which leads it to be used in Kaggle competitions widely.

Gradient Boosting builds a model sequentially, each iteration learning from the errors made by previous ones, thereby progressively reducing bias and variance. It is ideally suited to data with non-linear relationships among features, which makes it a strong candidate for this project.

The key steps of the Gradient Boosting algorithm are as follows:

1. Initialize with a Weak Model:

   The algorithm starts by fitting a simple model, typically a decision tree in this project. This initial model captures basic trends but leaves much room for improvement.

2. Calculate Residuals:

   The residuals represent the errors or negative gradients—what the model did not capture well. These residuals indicate how far off the model's predictions are from the true values.

3. Fit a New Model to the Residuals:

   A new model is trained to predict these residuals, effectively learning how to correct the mistakes made by the previous iteration. By doing this, each new model improves upon the weaknesses of the existing model ensemble.

4. Update the Model:

   The new model's predictions are added to the existing model using a learning rate parameter to determine how much influence the new model has. The learning rate controls the contribution of each new tree, which helps in preventing overfitting.

5. Repeat Until Convergence:

   This process repeats for a specified number of boosting rounds (n_estimators) or until additional iterations provide diminishing improvements. The final model is a collection of all the weak learners, which collectively make predictions.

Custom Implementation Details and Decision Tree as a part of Gradient Boosting:

Besides Gradient Boosting, I also implemented Decision Tree Regressors from scratch to be used as weak learners in the Gradient Boosting model. Each decision tree was designed with maximum depth to control complexity and avoid overfitting.

During each boosting round, these trees were trained on the residuals from the previous iteration, thereby learning from the errors of previous models. This iterative reduction of the residuals is what makes Gradient Boosting such a powerful approach for complex datasets.

Mathematical Expression and Explanation for the Decision Trees: Refer to [4]

The key concept behind Decision Trees is determining where to split the data. The following are the mathematical criteria used:

The **Gini Impurity** measures how often a randomly chosen element from the set would be incorrectly labeled if it were randomly labeled according to the distribution of labels in the dataset.

Formula for **Gini Impurity** of node t: $G(t) = 1 - \sum_{i=1}^{C} p_i^2$

where p_i is the proportion of class i instances in node t, and C is the number of classes.

Entropy is another measure of impurity, which comes from information theory. The idea is to measure the amount of disorder or randomness.

Formula for **Entropy** at node t: $H(t) = -\sum_{i=1}^{C} p_i \ \log_2 pi$

The **decision rule** is applied recursively to split the data at each node, based on minimizing the selected impurity measure.

The goal is to select a feature and threshold that minimizes the **weighted impurity** of the child nodes:

Impurity Reduction: $G(t) - \left(\frac{N_{left}}{N} G(t_{left}) + \frac{N_{right}}{N} G(t_{right})\right)$

where N_left and N_right are the number of samples in the left and right child nodes, respectively.

Mathematical Expression and Explanation for the Gradient Boosting: Refer to [5]

Gradient Boosting starts with an initial prediction, usually the mean value for regression tasks:

$F_0(x) = argmin_\gamma \sum_{i=1}^{N} L((y_i, \gamma)$ where L is the loss function and y_i are the observed values.

At each iteration m, calculate the residuals, which are essentially the negative gradient of the loss function with respect to the current model:

$r_i^{(m)} = -[\partial L((y_i, \mathrm{F(x_i)})/\partial \mathrm{F(x_i)}\ ]$ This residual represents what the model hasn't yet captured, indicating the direction in which the model needs to improve.

Now, fit a decision tree to the residuals. This Decision Tree becomes the new weak learner that helps to minimize the errors from the previous round. The weak learner h_m(x) is trained to fit these residuals.
The current model F_m(x) is updated using the output from the weak learner, scaled by a learning rate v: $F_m(x) = F_{m-1}(x) + v * h_m(x)$

where ν is a hyperparameter that controls the step size typically between 0 and 1. It helps in preventing overfitting by limiting how much influence each new tree has on the final model.

The process is repeated for M iterations or until the model stops improving significantly:

$$F_M(x) = F_0(x) + \sum_{i=1}^{M} v * h_m(x)$$

The final model F_M(x) is a sum of all the previous models, combining all the weak learners to form a strong, predictive model.

Log Loss (for Classification) in GB: $L(y_i, \mathrm{F}(\mathrm{x_i})) = -[\mathrm{y_i} \log \; pi + (1 - y_i)\log(1 - pi)]$

where p_i is the predicted probability for the positive class.

In **Decision Trees**, key operations involve finding optimal splits by minimizing **impurity measures** like **Gini impurity or entropy** for classification, and **MSE** for regression, to create homogeneous groups at the leaves.

In **Gradient Boosting**, multiple weak learners (Decision Trees) are trained iteratively on the **residuals** of previous models. The final model F_m(x) is built by adding these new trees step-by-step, scaled by a **learning rate**.

Together, Decision Trees provide a simple predictive model, while Gradient Boosting sequentially combines them to adapt and correct errors, resulting in a powerful, accurate ensemble model.

See the figure below for visual representation:

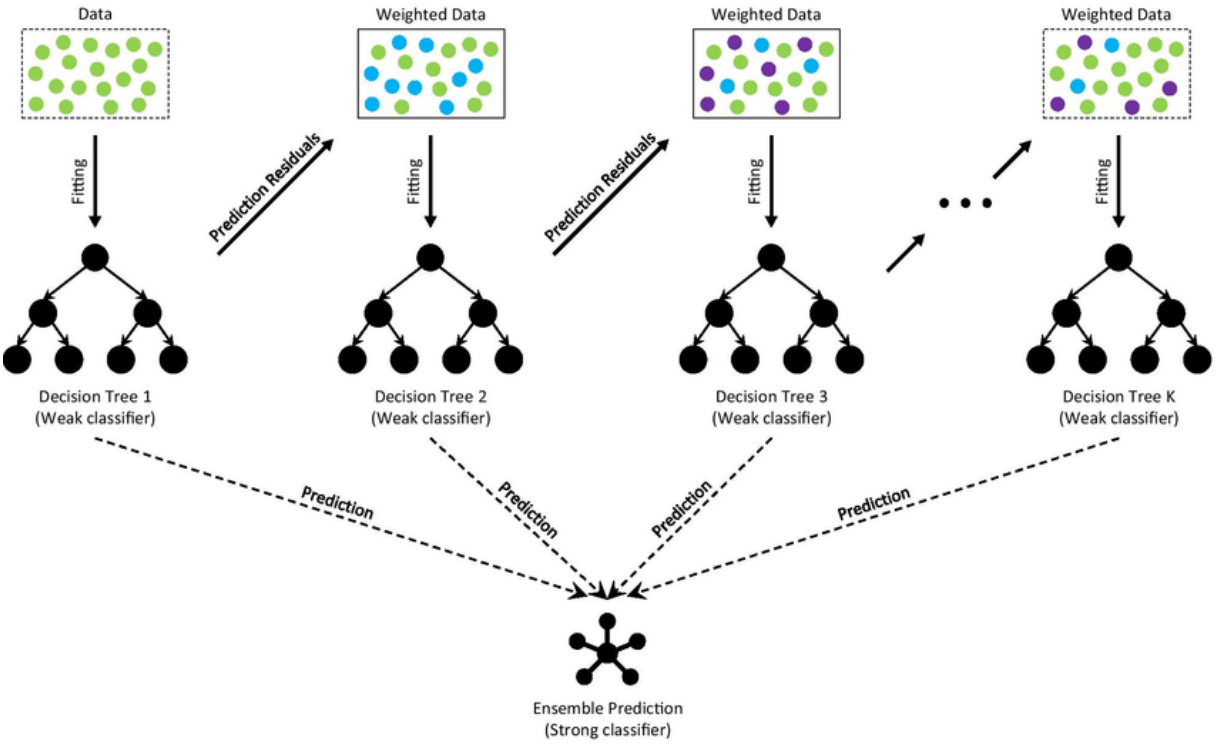


Figure 3: Visualization for Gradient Boosting using Decision Trees. [2]

3) Neural Networks (NN)

Neural networks are powerful tools for modeling intricate nonlinear relationships between input features and their corresponding outputs. These networks excel due to their ability to leverage parallel processing during both the training phase and when making predictions. This versatility makes them a suitable choice for many applications. Neural networks are composed of perceptrons organized into layers, which are connected sequentially from the input to the output. A perceptron itself is a composite unit comprising an internal linear computation followed by a nonlinear activation function By stacking these perceptrons in layers, the network can effectively capture and model complex nonlinear dependencies. Key parameters, such as the number of layers and the number of perceptrons per layer, are tunable hyperparameters that can be optimized for improved performance. Details about these optimizations and visualizations will be discussed in the upcoming sections.

Perceptron:

$$v = \sum_{i=0}^{d} w_i x_i \ , \ output = \phi(v)$$

Here, ϕ is the perceptron's activation function, which is typically nonlinear. For this neural network implementation, I employed the ReLU activation function for the hidden layer and the logistic activation function for the output layer. The specific definitions are as follows:

$$RELU(x) = x \ if \ x \geq 0 \ otherwise \ 0, \ \Phi(x) = \frac{1}{1+e^{-x}}$$

Network Architecture:

The network architecture consists of two hidden layers, resulting in three sets of weight matrices: input-to-layer1, layer1-to-output. The overall network output is computed as:

$$y = \Phi\left(W_{1-o} \cdot \left(RELU \cdot \left((W_{i-1} \cdot x_i)\right)\right)\right)$$

A prediction is then determined by comparing y to a specified threshold (which is 0.5 in my code):

$$y_{pred} = 1 \ if \ y \geq \ threshold = 1/2, \ otherwise \ 0$$

Training Procedure:

The network is trained using the backpropagation algorithm combined with mini-batch gradient descent. The mini-batch sizes are determined through k-fold cross-validation, with each batch containing n/k samples. The learning rate is scaled by a factor of 1/k. The loss function employed is the sum of squared errors. The backpropagation equations are as follows:

The local gradient at perceptron j in layer l:

$$\delta_j^l = -\frac{\partial J\left(y, y_{pred}\right)}{\partial v_j^l}, \ \frac{\partial J\left(y, y_{pred}\right)}{\partial w_{ij}^l} = \delta_j^l \cdot x_i^{l-1}$$

The backpropagation of gradients to layer l-1:

$$\delta_i^{l-1} = \sum_{j=1}^{d(l)} \delta_j^l \cdot w_{ij}^l \cdot \frac{\partial x_i^{l-1}}{\partial v_i^{l-1}}$$

Weight updates:

$$w_{ij}^{l} = w_{ij}^{l} - \frac{\eta}{k}\frac{\partial J(y\,, y_{pred})}{\partial w_{ij}^{l}}$$

Below is a figure representing a simple neural network system:

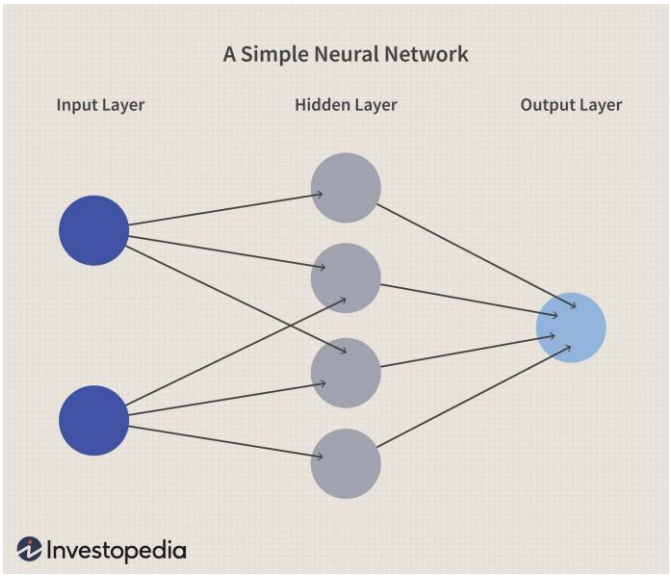


Figure 4: Visualization for a simple Nueral Network.

Note: After the First Report, Mr. Saday suggested me to use additionally a different dataset because my inital dataset was relatively easier to predict. Therefore, I have found a second dataset with a 40k samples and 16 features in a similar category about predicting whether a particle is psion or upsilon. Same dataprocessing steps are used also for this dataset. Therefore, I didn't mention all the procedures again. Until the neural network evaluation part my sections are the same as first report. But don't worry I will also re-evaluate and compare my algorithms and how they perform with this new dataset at the final evaluation and neural network part. And here is the corr. matrix for this dataset to explain briefly how it is distributed:

Figure 5: Correlation matrix for new dataset.

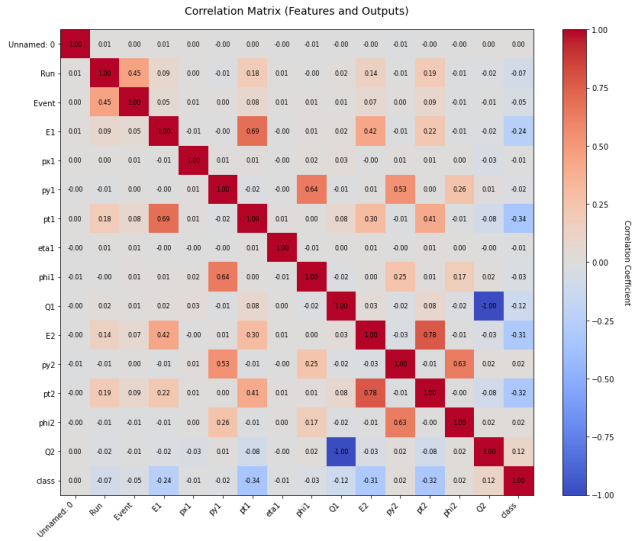


Table 3 and 4: Correlation matrix tables.

	Unnamed: 0	Run	Event	E1	px1	py1	pt1	eta1	phi1	Q1	E2	py2	pt2	phi2
Unnamed: 0	1.000000	0.005763	0.000546	0.006750	0.004041	-0.001658	0.003270	-0.004629	-0.007632	-0.002290	-0.001678	-0.010896	-0.001691	-0.003459
Run	0.005763	1.000000	0.449540	0.093411	0.002450	-0.008303	0.182071	0.005405	-0.002899	0.016390	0.136112	-0.006051	0.192656	-0.009841
Event	0.000546	0.449540	1.000000	0.051893	0.005005	0.001092	0.080566	0.005755	0.008111	0.005373	0.070202	0.000149	0.089774	-0.008776
E1	0.006750	0.093411	0.051893	1.000000	-0.006443	-0.001234	0.690425	-0.002145	0.011976	0.021517	0.423841	-0.013038	0.215519	-0.012590
px1	0.004041	0.002450	0.005005	-0.006443	1.000000	0.008419	0.008246	-0.002089	0.016692	0.034783	-0.000684	0.009658	0.005614	0.000619
py1	-0.001658	-0.008303	0.001092	-0.001234	0.008419	1.000000	-0.021684	-0.004596	0.639979	-0.008386	0.007504	0.533064	0.003412	0.255989
pt1	0.003270	0.182071	0.080566	0.690425	0.008246	-0.021684	1.000000	0.010848	0.002043	0.078513	0.302748	-0.009910	0.408427	-0.007898
eta1	-0.004629	0.005405	0.005755	-0.002145	-0.002089	-0.004596	0.010848	1.000000	-0.005252	0.002051	0.006143	-0.002003	0.010459	0.002953
phi1	-0.007632	-0.002899	0.008111	0.011976	0.016692	0.639979	0.002043	-0.005252	1.000000	-0.020040	0.004697	0.248457	0.009746	0.174580
Q1	-0.002290	0.016390	0.005373	0.021517	0.034783	-0.008386	0.078513	0.002051	-0.020040	1.000000	0.034189	-0.020123	0.081590	-0.022955
E2	-0.001678	0.136112	0.070202	0.423841	-0.000684	0.007504	0.302748	0.006143	0.004697	0.034189	1.000000	-0.025426	0.776199	-0.005243
py2	-0.010896	-0.006051	0.000149	-0.013038	0.009658	0.533064	-0.009910	-0.002003	0.248457	-0.020123	-0.025426	1.000000	-0.010197	0.631917
pt2	-0.001691	0.192656	0.089774	0.215519	0.005614	0.003412	0.408427	0.010459	0.009746	0.081590	0.776199	-0.010197	1.000000	-0.001075
phi2	-0.003459	-0.009841	-0.008776	-0.012590	0.000619	0.255989	-0.007898	0.002953	0.174580	-0.022955	-0.005243	0.631917	-0.001075	1.000000
Q2	0.002290	-0.016390	-0.005373	-0.021517	-0.034783	0.008386	-0.078513	-0.002051	0.020040	-1.000000	-0.034189	0.020123	-0.081590	0.022955
class	0.001271	-0.071719	-0.045501	-0.244860	-0.006420	-0.016627	-0.336340	-0.011177	-0.025373	-0.116429	-0.309122	0.022580	-0.322787	0.022975

Outcomes and Analysis of the Methods:

First, each method was individually optimized through **parameter tuning** using **k-fold cross-validation** to find the best configuration for each model. In the initial analysis, I identified the optimal parameters for both methods. Next, I split the data into **train and test sets** and trained each model using the same training data, evaluating their performance on the identical test set. This approach allowed to fairly compare and determine which method performed the best.

K-Fold Cross Validation

Cross-validation offers valuable insights into how well a model will perform on unseen data, providing a more reliable evaluation of its predictive accuracy. Therefore, I used k-fold cross-validation to identify the optimal parameters for each method. The entire dataset was used for cross-validation with fold number of **5** for the time complexity I didn't use higher k numbers. In the KNN algorithm, the first step involves dividing the dataset into k parts. In the second step, k-1 parts are used for training while 1 part is used for testing, and the accuracy is recorded. The third step involves repeating this process k times, each time shifting the test part. Finally, the average accuracy across all combinations is calculated. See the figure below for visual representation.

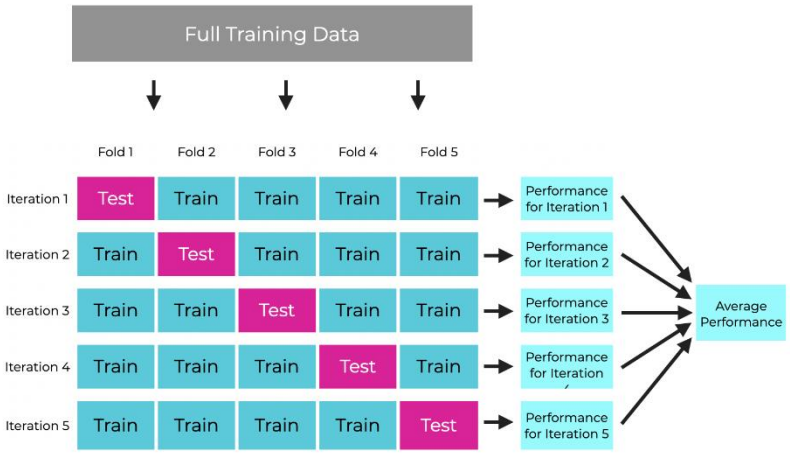


Figure 6: Visual representation of K-FOLD Cross Validation. [3]

1) K- Nearest Neighbors (KNN)

The k-NN algorithm's accuracy was evaluated using k-fold cross-validation across different values of k (number of neighbors), and the accuracy vs. k graph revealed that k=1 provided the highest accuracy, making it the optimal choice for the remainder of the project. Subsequently, an accuracy vs. threshold graph suggested 0.55 as the best probability threshold for classification. However, since the algorithm used majority voting instead of probability-based classification, the threshold concept applies only conceptually, reflecting the ratio of votes within the k-nearest neighbors rather than explicit probabilities. Classification is determined by majority voting among k-neighbors, and additional considerations, such as tie-breaking rules, ensure robust performance. This approach was supported by the observed data and methodology, aligning k=1 and majority voting as the key elements for achieving optimal accuracy. Where, for cross validation k = 5 is chosen. And problem was that since knn was slow, I developed a new knn using sorting algorithm to make it faster by priority queueing.

```
Num_k: 1
Accuracy scores for each fold: [0.978, 0.986, 0.979, 0.978, 0.973]
Mean accuracy: 0.9788
Num_k: 2
Accuracy scores for each fold: [0.968, 0.965, 0.968, 0.975, 0.968]
Mean accuracy: 0.9687999999999999
Num_k: 3
Accuracy scores for each fold: [0.98, 0.963, 0.969, 0.983, 0.97]
Mean accuracy: 0.9730000000000001
Num_k: 4
Accuracy scores for each fold: [0.969, 0.964, 0.954, 0.963, 0.973]
Mean accuracy: 0.9645999999999999
Num_k: 5
Accuracy scores for each fold: [0.969, 0.965, 0.965, 0.966, 0.972]
Mean accuracy: 0.9673999999999999
Num_k: 6
Accuracy scores for each fold: [0.96, 0.955, 0.953, 0.957, 0.96]
Mean accuracy: 0.9570000000000001
Num_k: 7
Accuracy scores for each fold: [0.948, 0.957, 0.963, 0.961, 0.955]
Mean accuracy: 0.9568
Num_k: 8
Accuracy scores for each fold: [0.944, 0.955, 0.95, 0.937, 0.955]
Mean accuracy: 0.9482000000000002
Num_k: 9
Accuracy scores for each fold: [0.95, 0.948, 0.948, 0.952, 0.956]
Mean accuracy: 0.9507999999999999
Num_k: 10
Accuracy scores for each fold: [0.93, 0.947, 0.929, 0.931, 0.942]
Mean accuracy: 0.9358000000000001
```

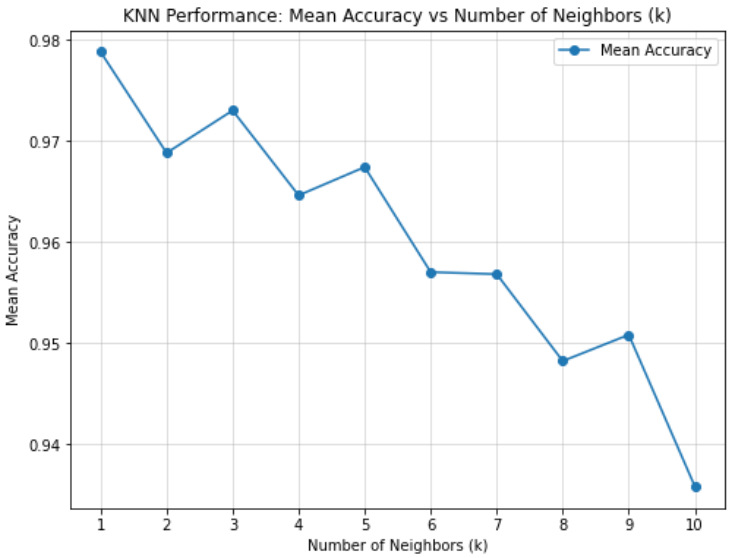


Figure 7: Accuricies for different neighboor numbers table and the plot.

In conclusion, the highest accuracy achieved was 97.88% with 1 nearest neighbor and a threshold of 0.55. The execution time for each iteration of the k-fold cross-validation, with 1 nearest neighbor and 5 folds, was 176.8 seconds for 20k samples. Despite utilizing vectorized operations, the k-fold cross-validation process still required significant computational time. To expedite the process, a subset of 20,000 data points, evenly sampled from the full dataset of 130,000, was used. See the figure below for the confusion matrix for the best parameter for 5K test data.

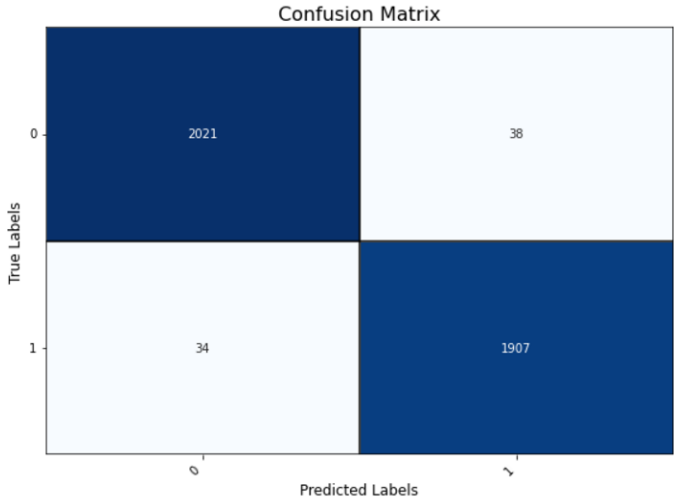


Figure 8: Confusion matrix for k = 1 for knn.

```
Precision per class: {0: 0.9834549878345499, 1: 0.9804627249357326}
Recall per class: {0: 0.9815444390480816, 1: 0.9824832560535807}
F1-Score per class: {0: 0.982498784637822, 1: 0.9814719505918682}
```

Figure 9: F1-Score for the knn.

2) Gradient Boosting Using Decision Trees

The gradient boosting method was employed to further enhance the model's performance, focusing on optimizing its hyperparameters for the best results with k cross validation where k=5. After an extensive hyperparameter tuning process, the optimal parameters were determined to be n_estimators=10, and learning_rate=0.1 and max_depth=6. These settings strike a balance between model complexity and training efficiency, ensuring a robust and generalized model. Gradient boosting leverages its iterative approach to correct errors from previous iterations, resulting in high accuracy and minimized overfitting. Unlike k-NN, which required optimization of the sorting process for speed, gradient boosting natively supports efficient computations and works well even with relatively large datasets. This combination of carefully tuned hyperparameters and gradient boosting's inherent strengths ensured that the model performed effectively on the given dataset. Eventually for 20k data sample 35.62 seconds took the calculation and accuracy of %99.04 was achieved with the best parameters. It was observed that GB is significantly good >%98 across all hyperparameters and way faster than the KNN. See the figures below for the results.

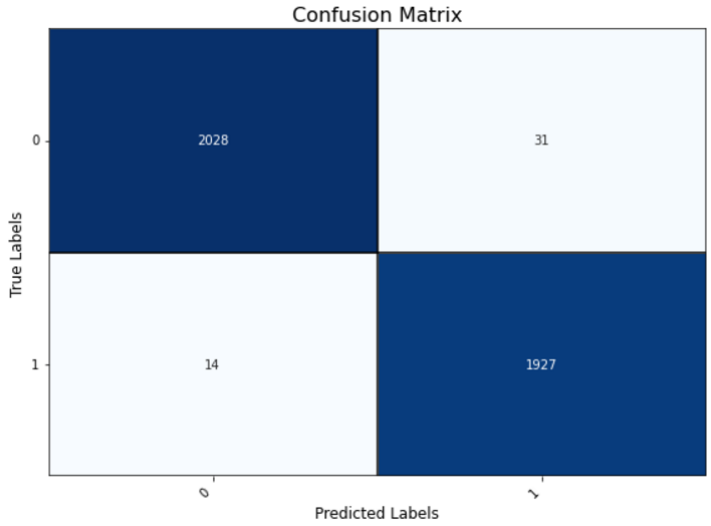


Figure 10: Confusion matrix for GB with the best parameters.

```
Precision per class for GB: {0: 0.9931439764936337, 1: 0.984167517875383}
Recall per class for GB: {0: 0.9849441476444876, 1: 0.992787223080862}
F1-Score per class for GB: {0: 0.9890270665691295, 1: 0.9884585791228521}
```

Figure 11: F1-Score for the GB.

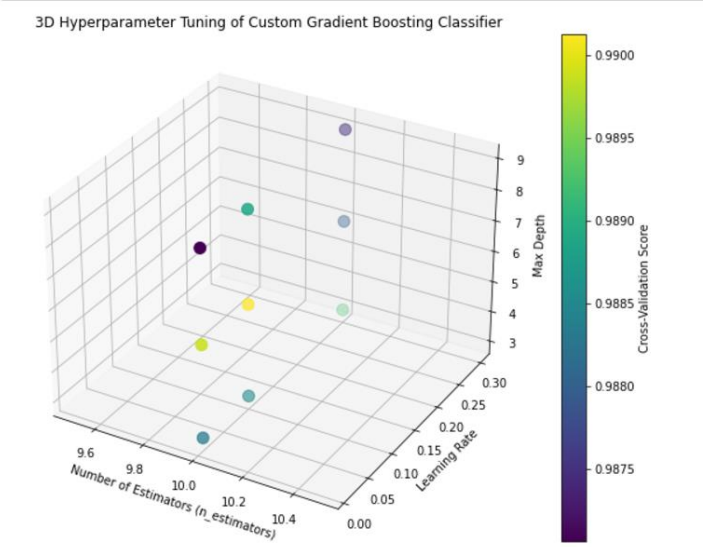


Figure 12: 3-D plot of Hyperparameter Tuning of GB where n_estimators is fixed and learning rate and max depth are the changing variables.

In conclusion, The gradient boosting method, employing decision trees as base learners, achieved a significant improvement in performance, with the highest accuracy of 99.04% on a 20,000-sample dataset using the best hyperparameters: n_estimators=10 and learning_rate=0.1, and max_depth=6. With 5-fold cross-validation, the execution time was 35.62 seconds, showcasing a considerable computational advantage over k-NN, which required more time for equivalent tasks. Moreover, gradient boosting demonstrated consistently high accuracy, exceeding 98% across all tested hyperparameter combinations, highlighting its robustness and efficiency. These results, visualized in Figures 10 and 11 (confusion matrix and F1-Score, respectively), underscore the method's suitability for large datasets and its ability to balance computational speed with predictive performance, outperforming k-NN in both metrics.

3) Neural Network

Read the **"Note".** First, I used the **second** dataset. Results for the **second** dataset:

Firstly, I did the weight initialization simply using standart normal distribution scaled with 1/100. I used 5-fold cross-validation to determine the optimal values for the hyperparameters: hidden layer dimension, learning rate, and batch size, which are applied during the weight training process. I tried total 45 different iterations for hyperparameter tuning. And also **100 epochs** are constantly used. After hyperparameter optimization best parameters and the accuracy is found as **%89.0** shown below:

```
Best Accuracy: 0.8900 with Parameters: Learning Rate=0.01, Layers=4, Batch Size=16
```

```
for lr in [0.001, 0.01, 0.1]:
    for hidden_units in [2,4,8,16,32]:
        for batch_size in [4, 8, 16]:
```

Below plots show how the accuracy behaves based on the hyperparameters (lr, batch_size, units):

Figure 13: Accuracy vs all hyper parameters

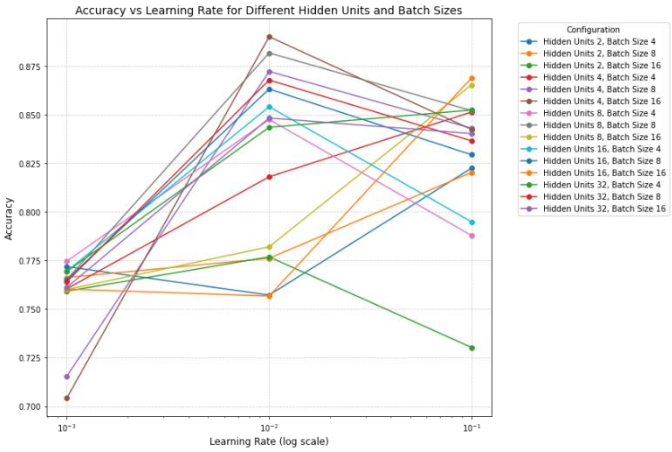


Figure 14: Accuracy vs Batchsize

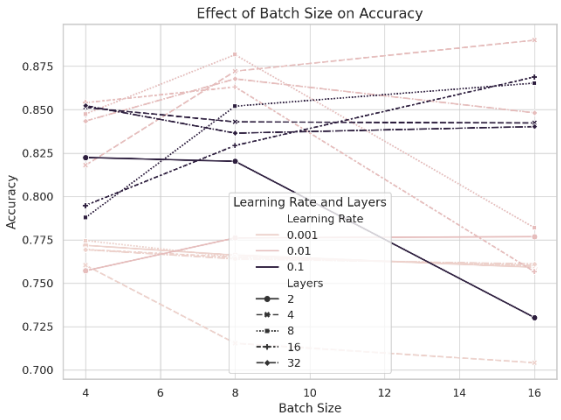


Figure 15: Accuracy vs number of layers

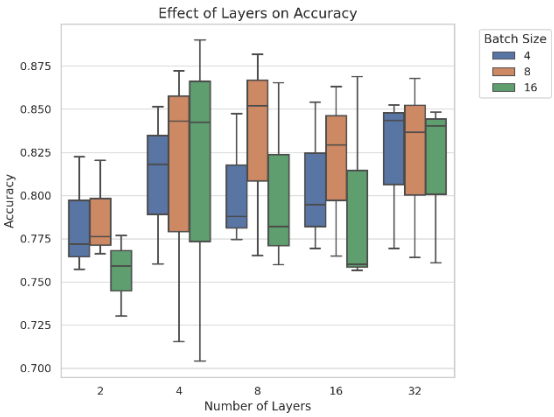
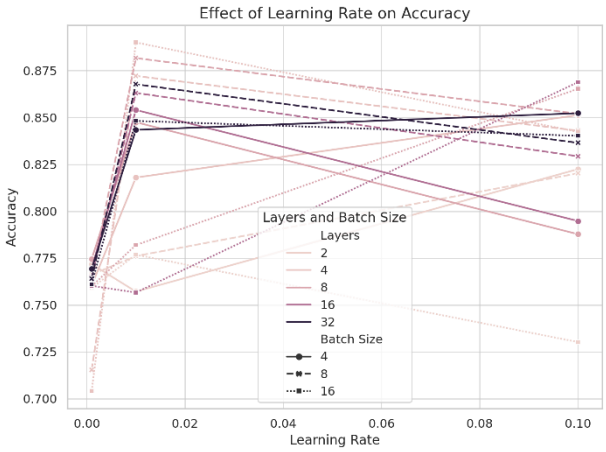
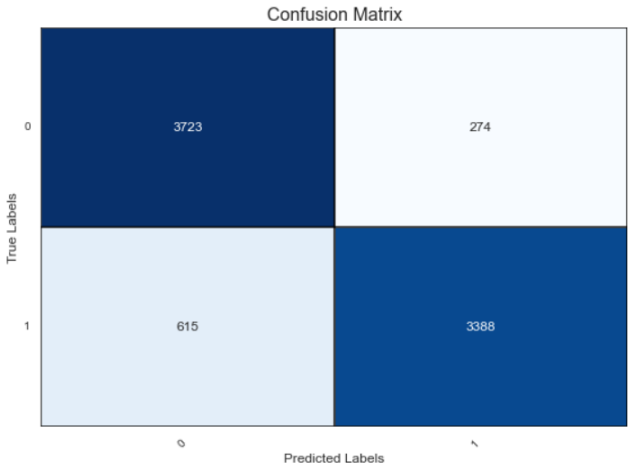


Figure 16: Accuracy vs Learning Rate



The analysis of hyperparameter effects shows that smaller learning rates (e.g., 0.001) consistently yield higher accuracy, indicating better convergence during training. Models with 2 to 4 layers achieve a good balance between complexity and performance, while deeper networks show increased variability. Smaller batch sizes, particularly in the range of 4 to 8, enhance accuracy, likely due to improved gradient estimation, though they may slightly increase training time. Overall, a learning rate of 0.001, 2-4 layers, and batch sizes of 4-8 are recommended for optimal accuracy, balancing performance with training efficiency. Below are the metrics of the best parameters.




```
Confusion Matrix:
[[3723  274]
 [ 615 3388]]
Class Labels: [0 1]
```

Figure 17 and 18: Confusion Matrixes

```
Precision per class: {0: 0.8582295988934993, 1: 0.9251774986346258}
Recall per class: {0: 0.9314485864398299, 1: 0.8463652260804396}
F1-Score per class: {0: 0.8933413317336533, 1: 0.8840182648401826}
```

Figure 19: F1 Score for NN based on second (new) dataset

Results for the first dataset for NN:

Similarly with above, I used the best hyperparameters found while using the first dataset and these are the evaluation metrics for the first dataset using NN.

```
Accuracy: 0.9901
```

```
Confusion Matrix:
[[13960   174]
 [   87 12262]]
Class Labels: [0 1]
```

```
Precision per class: {0: 0.9938065067274151, 1: 0.9860083628176263}
Recall per class: {0: 0.9876892599405689, 1: 0.9929548951332091}
F1-Score per class: {0: 0.9907384407934424, 1: 0.9894694371595724}
```

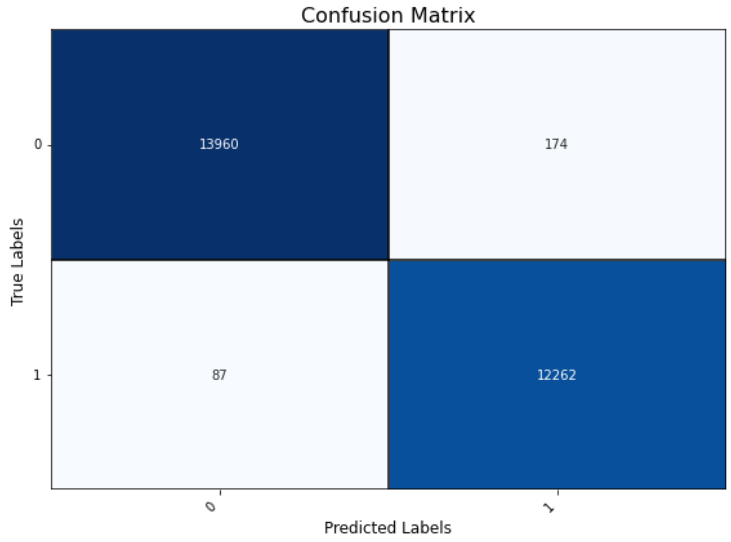


Figure 20-21-22-23: Performance metrics of the NN based on first (old) dataset

```
Elapsed time for NN execution: 12.7760 seconds
```

In conclusion, The neural network achieved 89.0% accuracy on the second dataset with optimized hyperparameters: learning rate of 0.001, batch sizes of 4-8, and 2-4 layers. Smaller learning rates improved convergence, while smaller batch sizes enhanced gradient estimation. Consistent results on the first dataset confirm the effectiveness of the chosen hyperparameters and the model's reliability.

Now I will compare all the methods in the preceeding section.

Comparison of Methods and Analysis and Comments

In fact, I have 4 different methods: KNN, Gradient Boosting, NN but also a Decision Tree that is used inside of Gradient Boosting. Therefore, In this section I compare those four algorithms results based on the first and the second dataset with best hyperparameters.

Below table shows how accuracies behave for each algorithm and each dataset.

Algorithm/Dataset	First (old)	Second (new)
KNN	%97.88	%57.04
GRADIENT BOOSTING	%99.04	%83.65
NEURAL NETWORK	%99.01	%89.0
DECISION TREE	%97	%81.8

Table 5: Accuracies vs the algorithms and the datasets

The comparison of methods highlights the strengths and weaknesses of KNN, Gradient Boosting, Neural Networks (NN), and Decision Trees across two datasets:

- **First Dataset**: Gradient Boosting and NN achieved nearly identical top-tier accuracy (~99%), showcasing robustness. KNN and Decision Trees also performed well (~97%) but lagged slightly.
- **Second Dataset**: NN excelled with 89.0% accuracy, followed by Gradient Boosting (83.65%) and Decision Trees (81.8%). KNN struggled significantly (57.04%) due to the dataset's complexity.

Method Strengths

- **Neural Networks**: Strong performance across datasets, adept at modeling complex patterns.
- **Gradient Boosting**: Competitive on simpler datasets, with effective handling of moderate complexity.
- **KNN**: Suitable for simpler, smaller datasets but struggles with high-dimensional or noisy data.
- **Decision Trees**: Moderate standalone performance but lacks the ensemble power of Gradient Boosting.

Insights

- Neural Networks and Gradient Boosting offer the best balance of accuracy and generalization.
- KNN's computational demands and sensitivity to complexity make it less favorable for challenging datasets.
- Decision Trees perform well but are more effective when used within an ensemble like Gradient Boosting.

Finally, below plots show the how accuracy behaves based on the algorithms:

 Bar Chart for Accuracy Comparison: Displays the accuracy of each algorithm on both datasets, unchanged from before.

 Scatter Plot for Execution Time vs. Accuracy: Reflects the updated execution time for Neural Networks (**12.76 seconds**) alongside the accuracies for all methods. Labels identify each algorithm, emphasizing the balance between computational efficiency and performance.

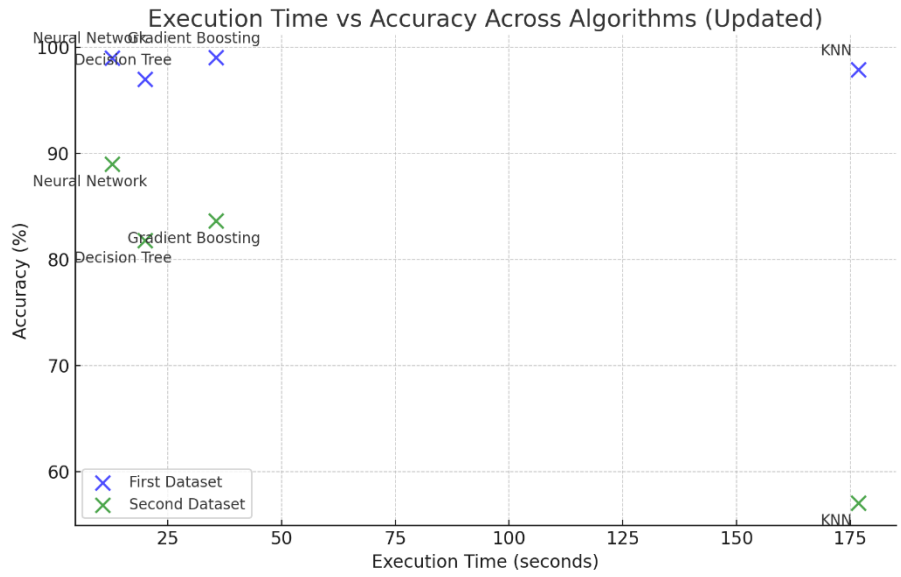


Figure 24: Accuracy vs Execution Time plot for each algorithm

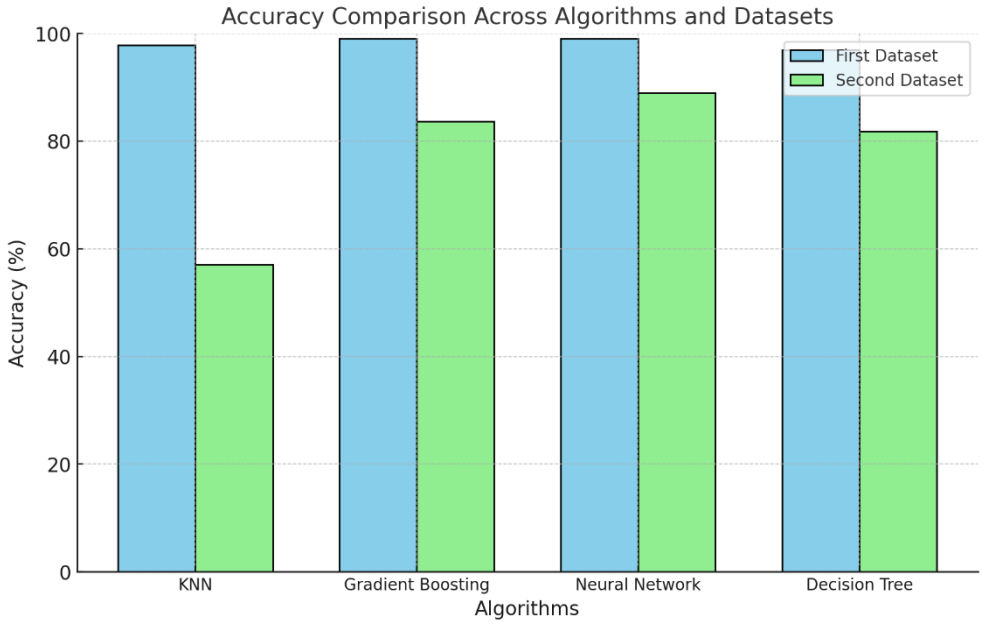


Figure 25: Accuracy vs Algorithms plot for each dataset

References

- Medium. "K-Nearest Neighbor," [Online]. Available: https://medium.com/swlh/k-nearest-neighbor-ca2593d7a3c4.
- ResearchGate. "The Architecture of Gradient Boosting Decision Tree," [Online]. Available: https://www.researchgate.net/figure/The-architecture-of-Gradient-Boosting-Decision-Tree_fig2_356698772.
- SharpSight Labs. "Cross-Validation Explained," [Online]. Available: https://www.sharpsightlabs.com/blog/cross-validation-explained/.
- Gareth James, Daniela Witten, Trevor Hastie, and Robert Tibshirani. *An Introduction to Statistical Learning with Applications in R*. Springer, 2014.
- Friedman, J. H. "Greedy Function Approximation: A Gradient Boosting Machine." *Annals of Statistics*, vol. 29, 2001, pp. 1189–1232.

Appendix

#LIBRARY IMPORTING

```
import numpy as np

import pandas as pd

import seaborn as sns

import matplotlib.pyplot as plt

import time as tm

from sklearn.model_selection import train_test_split

from collections import Counter

import heapq

%matplotlib inline
```

#DATA HANDLING

```
zoo = pd.read_csv("GalaxyZoo1_DR_table2.csv")

zoo_clean = zoo[zoo["UNCERTAIN"]==0]

zoo_clean.drop(columns=["UNCERTAIN"], inplace = True)

elliptical_zero = zoo_clean[zoo_clean["ELLIPTICAL"] == 0]

rows_to_drop = elliptical_zero.sample(n=120000, random_state=42).index

zoo_clean = zoo_clean.drop(index=rows_to_drop).reset_index(drop=True)

print(zoo_clean["ELLIPTICAL"].value_counts())

zoo_clean.drop(columns=["OBJID", "RA", "DEC"], inplace=True) #drops the columns related to the location of the galaxies and ids not important for prediction from my physics knowledge. just a identification of this specific galaxy.

feature_columns = ['NVOTE', 'P_CW', 'P_ACW', 'P_EDGE', 'P_DK', 'P_MG']

output_columns = ['SPIRAL', 'ELLIPTICAL']
```

```
data_array = zoo_clean[feature_columns + output_columns].to_numpy()

columns = feature_columns + output_columns

def correlation_coefficient(x, y):

    x_mean = np.mean(x)

    y_mean = np.mean(y)

    numerator = np.sum((x - x_mean) * (y - y_mean))

    denominator = np.sqrt(np.sum((x - x_mean) ** 2)) * np.sqrt(np.sum((y - y_mean) **
2))

    return numerator / denominator if denominator != 0 else 0

n_features = len(columns)

corr_matrix = np.zeros((n_features, n_features))

for i in range(n_features):

    for j in range(n_features):

        corr_matrix[i, j] = correlation_coefficient(data_array[:, i], data_array[:, j])

plt.figure(figsize=(12, 10))

plt.imshow(corr_matrix, cmap='coolwarm', interpolation='none')

plt.colorbar(label="Correlation Coefficient")

plt.title("Correlation Matrix (Features and Outputs)")

plt.xticks(range(n_features), columns, rotation=90)

plt.yticks(range(n_features), columns)

plt.tight_layout()

plt.show()

def standardize_data(X):

    means = np.mean(X, axis=0)

    stds = np.std(X, axis=0)

    stds[stds == 0] = 1
```

```
X_standardized = (X - means) / stds

return X_standardized, means, stds

n_features = 6 + 2

columns = ['NVOTE', 'P_CW', 'P_ACW', 'P_EDGE', 'P_DK', 'P_MG', 'SPIRAL',
'ELLIPTICAL']

plt.figure(figsize=(12, 10))

plt.imshow(corr_matrix, cmap='coolwarm', interpolation='none', aspect='auto')

cbar = plt.colorbar()

cbar.set_label("Correlation Coefficient", rotation=270, labelpad=20)

plt.grid(False)

plt.title("Correlation Matrix (Features and Outputs)", fontsize=14, pad=20)

plt.xticks(range(n_features), columns, rotation=45, ha="right", fontsize=10)

plt.yticks(range(n_features), columns, fontsize=10)

for i in range(n_features):

    for j in range(n_features):

        value = corr_matrix[i, j]

        plt.text(j, i, f"{value:.2f}", ha="center", va="center", fontsize=8, color="black")

plt.tight_layout()

plt.show()

corr_matrix_df = pd.DataFrame(corr_matrix, index=columns, columns=columns)

print(corr_matrix_df)

reduced_corr_matrix_df = corr_matrix_df.drop(index=feature_columns,
columns=output_columns)

print(reduced_corr_matrix_df)

features_array = zoo_clean[feature_columns].to_numpy()

outputs_array = zoo_clean["ELLIPTICAL"].to_numpy() #Take only the ELLIPTICAL, if it is
0 it is SPIRAL if it is 1 it is ELLIPTICAL
```

```
print(features_array, outputs_array)
```

#KNN SIMPLE

```
class KNNClassifier:
    def __init__(self, k=3):
        self.k = k
    def fit(self, X, y):
        self.X_train = X
        self.y_train = y
    def predict(self, X_test):
        predictions = [self._predict_single(x) for x in X_test]
        return np.array(predictions)
    def _predict_single(self, x):
        distances = [np.linalg.norm(x - x_train) for x_train in self.X_train]
        k_indices = np.argsort(distances)[:self.k]
        k_nearest_labels = [self.y_train[i] for i in k_indices]
        most_common = Counter(k_nearest_labels).most_common(1)
        return most_common[0][0]
```

#KNN WITH PRIORITY QUEUING

```
class KNNClassifier_Pq:
    def __init__(self, k=3):
        self.k = k
    def fit(self, X, y):
        self.X_train = X
        self.y_train = y
    def predict(self, X_test):
        predictions = [self._predict_single(x) for x in X_test]
        return np.array(predictions)
    def _predict_single(self, x):
        max_heap = []
        for idx, x_train in enumerate(self.X_train):
            distance = np.linalg.norm(x - x_train)
            if len(max_heap) < self.k:
                heapq.heappush(max_heap, (-distance, idx))
            else:
                heapq.heappushpop(max_heap, (-distance, idx))
        k_indices = [idx for _, idx in max_heap]
        k_nearest_labels = [self.y_train[i] for i in k_indices]
        most_common = Counter(k_nearest_labels).most_common(1)
        return most_common[0][0]
```

```
X_train, X_test, y_train, y_test = train_test_split(
    features_array[:20000], outputs_array[:20000], test_size=0.2, random_state=42
)
y_train = y_train.ravel()
print("X_train shape:", X_train.shape)
print("y_train shape:", y_train.shape)

X_train_new = X_train[:]
y_train_new = y_train[:]
X_test_new = X_test[:]
y_test_new = y_test[:]
start_time = tm.time()
knn = KNNClassifier_Pq(k=1)
knn.fit(X_train_new, y_train_new)
predictions = knn.predict(X_test_new)
end_time = tm.time()
elapsed_time = end_time - start_time
print(f"Elapsed time for k-NN execution: {elapsed_time:.4f} seconds")

accuracy = np.mean(predictions == y_test_new)
print(f"Accuracy: {accuracy * 100:.2f}%")
```

```
def compute_confusion_matrix(y_true, y_pred):
    class_labels = np.unique(np.concatenate((y_true, y_pred)))
    n_classes = len(class_labels)
    conf_matrix = np.zeros((n_classes, n_classes), dtype=int)
    label_to_index = {label: index for index, label in enumerate(class_labels)}
    for true_label, pred_label in zip(y_true, y_pred):
        i = label_to_index[true_label]
        j = label_to_index[pred_label]
        conf_matrix[i][j] += 1
    return conf_matrix, class_labels


conf_matrix, class_labels = compute_confusion_matrix(y_test_new, predictions)
print("Confusion Matrix:")
print(conf_matrix)
print("Class Labels:", class_labels)
conf_matrix_df = pd.DataFrame(conf_matrix, index=class_labels, columns=class_labels)


plt.figure(figsize=(8, 6))
ax = sns.heatmap(conf_matrix_df, annot=True, fmt="d", cmap="Blues", linewidths=0.5,
cbar=False)
plt.title("Confusion Matrix", fontsize=16)
plt.xlabel("Predicted Labels", fontsize=12)
plt.ylabel("True Labels", fontsize=12)
```

```
plt.xticks(np.arange(len(class_labels)) + 0.5, class_labels, rotation=45, ha="right",
fontsize=10)

plt.yticks(np.arange(len(class_labels)) + 0.5, class_labels, rotation=0, fontsize=10)

ax.hlines(np.arange(len(class_labels) + 1), *ax.get_xlim(), colors="black", linewidth=1.5)

ax.vlines(np.arange(len(class_labels) + 1), *ax.get_ylim(), colors="black", linewidth=1.5)

plt.tight_layout()

plt.show()

def compute_precision_recall_f1(conf_matrix):

    n_classes = conf_matrix.shape[0]

    precision_dict = {}

    recall_dict = {}

    f1_dict = {}

    for i in range(n_classes):

        TP = conf_matrix[i, i]

        FP = conf_matrix[:, i].sum() - TP

        FN = conf_matrix[i, :].sum() - TP

        # Precision: TP / (TP + FP)

        precision = TP / (TP + FP) if (TP + FP) > 0 else 0.0

        # Recall: TP / (TP + FN)

        recall = TP / (TP + FN) if (TP + FN) > 0 else 0.0

        # F1-Score: 2 * (Precision * Recall) / (Precision + Recall)

        f1 = 2 * precision * recall / (precision + recall) if (precision + recall) > 0 else 0.0

        precision_dict[i] = precision

        recall_dict[i] = recall

        f1_dict[i] = f1
```

```
    return precision_dict, recall_dict, f1_dict
def k_fold_cross_validation_knn(model_class, X, y, k=5, num_k=3):

    indices = np.arange(len(X))
    np.random.shuffle(indices)
    X = X[indices]
    y = y[indices]

    fold_size = len(X) // k
    scores = []

    for fold in range(k):
        start = fold * fold_size
        end = start + fold_size
        X_val = X[start:end]
        y_val = y[start:end]
        X_train = np.concatenate((X[:start], X[end:]), axis=0)
        y_train = np.concatenate((y[:start], y[end:]), axis=0)

        model = model_class(num_k)
        model.fit(X_train, y_train)
        predictions = model.predict(X_val)
        accuracy = np.mean(predictions == y_val)
        scores.append(accuracy)
return scores
```

```
k_scores = []

for i in range(1,11):

    scores = k_fold_cross_validation_knn(KNNClassifier_Pq, features_array[:5000], outputs_array[:5000], k=5, num_k=i)

    print("Num_k:", i)

    print("Accuracy scores for each fold:", scores)

    print("Mean accuracy:", np.mean(scores))

    k_scores.append(np.mean(scores))

num_k_values = list(range(1, 11))

plt.figure(figsize=(8, 6))

plt.plot(num_k_values, k_scores, marker='o', linestyle='-', label='Mean Accuracy')

plt.title("KNN Performance: Mean Accuracy vs Number of Neighbors (k)")

plt.xlabel("Number of Neighbors (k)")

plt.ylabel("Mean Accuracy")

plt.xticks(num_k_values)

plt.grid(alpha=0.5)

plt.legend()

plt.show()

precision_dict, recall_dict, f1_dict = compute_precision_recall_f1(conf_matrix)

precision_scores = {class_labels[i]: precision_dict[i] for i in range(len(class_labels))}

recall_scores = {class_labels[i]: recall_dict[i] for i in range(len(class_labels))}

f1_scores = {class_labels[i]: f1_dict[i] for i in range(len(class_labels))}

print("Precision per class:", precision_scores)

print("Recall per class:", recall_scores)

print("F1-Score per class:", f1_scores)
```

#DECISION TREE

```
class DecisionTreeRegressorScratch:
    def __init__(self, max_depth=3, min_samples_split=2):
        self.max_depth = max_depth
        self.min_samples_split = min_samples_split
        self.tree = None

    def fit(self, X, y):
        '''Fit the decision tree to the data'''
        self.tree = self._build_tree(X, y, depth=0)

    def predict(self, X):
        '''Predict values for the given input data'''
        return np.array([self._predict_row(x, self.tree) for x in X])

    def _build_tree(self, X, y, depth):
        '''Recursive function to build the decision tree'''
        # Stopping criteria
        if (depth >= self.max_depth) or (len(y) < self.min_samples_split) or (len(np.unique(y)) == 1):
            return {'value': np.mean(y)}

        # Find the best split
        feature_idx, threshold = self._best_split(X, y)
```

```
    if feature_idx is None:
        return {'value': np.mean(y)}

    left_indices = X[:, feature_idx] <= threshold
    right_indices = X[:, feature_idx] > threshold

    # Recursively build left and right branches
    left_subtree = self._build_tree(X[left_indices], y[left_indices], depth + 1)
    right_subtree = self._build_tree(X[right_indices], y[right_indices], depth + 1)

    # Return the decision node
    return {
        'feature_idx': feature_idx,
        'threshold': threshold,
        'left': left_subtree,
        'right': right_subtree
    }

def _best_split(self, X, y):
    '''Find the best split for a node'''
    best_feature, best_threshold = None, None
    best_mse = float('inf')
    n_samples, n_features = X.shape

    # Loop over all features
```

```
    for feature_idx in range(n_features):
        thresholds = np.unique(X[:, feature_idx])

        # Try each threshold as a split point
        for threshold in thresholds:
            left_indices = X[:, feature_idx] <= threshold
            right_indices = X[:, feature_idx] > threshold

            if len(y[left_indices]) == 0 or len(y[right_indices]) == 0:
                continue

            # Calculate the MSE for this split
            mse = self._calculate_mse(y[left_indices], y[right_indices])

            # Update the best split if this one is better
            if mse < best_mse:
                best_mse = mse
                best_feature = feature_idx
                best_threshold = threshold

    return best_feature, best_threshold

def _calculate_mse(self, left_y, right_y):
    '''Calculate Mean Squared Error for a given split'''
    left_mse = np.var(left_y) * len(left_y) if len(left_y) > 0 else 0
```

```
    right_mse = np.var(right_y) * len(right_y) if len(right_y) > 0 else 0
    total_mse = (left_mse + right_mse) / (len(left_y) + len(right_y))
    return total_mse

def _predict_row(self, x, tree):
    '''Predict a value for a single data point'''
    # If the node is a leaf node, return its value
    if 'value' in tree:
        return tree['value']
    # Otherwise, traverse the tree
    feature_idx = tree['feature_idx']
    threshold = tree['threshold']
    if x[feature_idx] <= threshold:
        return self._predict_row(x, tree['left'])
    else:
        return self._predict_row(x, tree['right'])
def update_leaf_values(self, X, negative_gradients, hessians):
    '''Update the terminal nodes based on negative gradients and Hessians'''
    def _traverse_and_update(node, X, negative_gradients, hessians):
        if 'value' in node:  # If it's a leaf node
            indices = self._get_samples_in_leaf(node, X)
            if len(indices) > 0:
                negative_gradients_in_leaf = negative_gradients[indices]
                hessians_in_leaf = hessians[indices]
                if np.sum(hessians_in_leaf) != 0:
```

```
                node['value'] = np.sum(negative_gradients_in_leaf) / np.sum(hessians_in_leaf)
            else:
                # Recur for left and right branches
                feature_idx = node['feature_idx']
                threshold = node['threshold']
                left_indices = X[:, feature_idx] <= threshold
                right_indices = X[:, feature_idx] > threshold
                _traverse_and_update(node['left'], X[left_indices], negative_gradients[left_indices], hessians[left_indices])
                _traverse_and_update(node['right'], X[right_indices], negative_gradients[right_indices], hessians[right_indices])
        _traverse_and_update(self.tree, X, negative_gradients, hessians)
    def _get_samples_in_leaf(self, node, X):
        '''Find samples that belong to a specific leaf node'''
        indices = []
        for i, x in enumerate(X):
            current_node = self.tree  # Start from the root
            while 'value' not in current_node:  # Traverse until a leaf node is reached
                feature_idx = current_node['feature_idx']
                threshold = current_node['threshold']
                if x[feature_idx] <= threshold:
                    current_node = current_node['left']
                else:
                    current_node = current_node['right']
            if current_node is node:
                indices.append(i)
```

```
        return indices
```

#GRADIENT BOOSTING

```
class CustomGradientBoostingClassifier:


    def __init__(self, n_estimators=100, learning_rate=0.1, max_depth=3):
        self.n_estimators = n_estimators
        self.learning_rate = learning_rate
        self.max_depth = max_depth


    def fit(self, X, y):
        """
        Fit the Gradient Boosting model to the provided training data.
        """
        self.num_classes = len(np.unique(y))
        y_encoded = self._one_hot_encode(y)


        initial_raw_predictions = np.zeros_like(y_encoded)
        class_probs = self._compute_softmax(initial_raw_predictions)


        self.models = []


        for i in range(self.n_estimators):
            trees_per_class = []


            for class_idx in range(self.num_classes):
```

```
            residuals = self._compute_residuals(y_encoded[:, class_idx], class_probs[:,
class_idx])

            hessians = self._compute_hessian(class_probs[:, class_idx])



            weak_learner = DecisionTreeRegressorScratch(max_depth=self.max_depth,
min_samples_split=2)

            weak_learner.fit(X, residuals)



            self._adjust_terminal_nodes(weak_learner, X, residuals, hessians)



            initial_raw_predictions[:, class_idx] += self.learning_rate *
weak_learner.predict(X)



            trees_per_class.append(weak_learner)



        class_probs = self._compute_softmax(initial_raw_predictions)

        self.models.append(trees_per_class)



    def _one_hot_encode(self, y):



        if isinstance(y, pd.Series):

            y = y.values

        y = np.array(y)



        n_classes = np.max(y) + 1



        y_ohe = np.zeros((y.shape[0], n_classes))
```

```
    y_ohe[np.arange(y.shape[0]), y] = 1

    return y_ohe

def _compute_residuals(self, y_true, class_prob):
    """Calculate residuals (negative gradients)."""
    return y_true - class_prob

def _compute_hessian(self, class_prob):
    """Calculate the hessian, representing the second derivative of the loss function."""
    return class_prob * (1 - class_prob)

def _compute_softmax(self, raw_preds):
    """Compute softmax transformation on raw predictions to obtain probabilities."""
    exp_vals = np.exp(raw_preds)
    exp_sum = np.sum(exp_vals, axis=1, keepdims=True)
    return exp_vals / exp_sum

def _adjust_terminal_nodes(self, tree, X, residuals, hessians):
    """
    Adjust the values at terminal nodes of the decision tree based on residuals and hessians.

    """
    def _traverse_and_update(node, X, residuals, hessians):
```

```
            # Check if we are at a leaf node
            if 'value' in node:
                leaf_indices = self._get_indices_in_leaf(node, X)
                if len(leaf_indices) > 0:
                    residuals_leaf = residuals[leaf_indices]
                    hessians_leaf = hessians[leaf_indices]
                    if np.sum(hessians_leaf) != 0:
                        node['value'] = np.sum(residuals_leaf) / np.sum(hessians_leaf)
            else:
                # Recursively handle children (left and right branches)
                feature_idx = node['feature_idx']
                threshold = node['threshold']

                left_indices = X[:, feature_idx] <= threshold
                right_indices = X[:, feature_idx] > threshold

                _traverse_and_update(node['left'], X[left_indices], residuals[left_indices], hessians[left_indices])
                _traverse_and_update(node['right'], X[right_indices], residuals[right_indices], hessians[right_indices])

        # Start from the root node of the tree
        _traverse_and_update(tree.tree, X, residuals, hessians)

    def predict_proba(self, X):
        """
```

```
        Predict class probabilities for given input data.

        """
        initial_raw_predictions = np.zeros((X.shape[0], self.num_classes))

        for model_round in self.models:
            for class_idx, weak_learner in enumerate(model_round):
                initial_raw_predictions[:, class_idx] += self.learning_rate *
weak_learner.predict(X)

        return self._compute_softmax(initial_raw_predictions)

    def predict(self, X):
        """
        Predict class labels for given input data.

        """
        class_probs = self.predict_proba(X)
        return np.argmax(class_probs, axis=1)

    def _get_indices_in_leaf(self, node, X):
        """Find indices of samples that belong to a given leaf node."""
        indices = []

        for i, x in enumerate(X):
            current_node = node
```

```
    while 'value' not in current_node:
        feature_idx = current_node['feature_idx']
        threshold = current_node['threshold']

        if x[feature_idx] <= threshold:
            current_node = current_node['left']
        else:
            current_node = current_node['right']

    if current_node is node:
        indices.append(i)

return indices
```

```
def k_fold_cross_validation_gbm(model_class, X, y, k=5, **model_params):

    indices = np.arange(len(X))
    np.random.shuffle(indices)
    X = X[indices]
    y = y[indices]

    fold_size = len(X) // k
    scores = []

    for fold in range(k):
        start = fold * fold_size
        end = start + fold_size

        X_val = X[start:end]
        y_val = y[start:end]
        X_train = np.concatenate((X[:start], X[end:]), axis=0)
        y_train = np.concatenate((y[:start], y[end:]), axis=0)
        model = model_class(**model_params)
        model.fit(X_train, y_train)
        predictions = model.predict(X_val)
        accuracy = np.mean(predictions == y_val)
        scores.append(accuracy)
```

```
    return scores
n_estimators_list = [10, 50, 100]
learning_rate_list = [0.01, 0.1, 0.3]
max_depth_list = [3, 6, 9]

best_params = None
best_score = -np.inf

results = []

for n_estimators in n_estimators_list:
    for learning_rate in learning_rate_list:
        for max_depth in max_depth_list:
            model_params = {
                'n_estimators': n_estimators,
                'learning_rate': learning_rate,
                'max_depth': max_depth
            }

            cv_scores = k_fold_cross_validation_gbm(CustomGradientBoostingClassifier, X_train_new, y_train_new, k=5, **model_params)
            avg_cv_score = np.mean(cv_scores)

            results.append({
                'n_estimators': n_estimators,
```

```
            'learning_rate': learning_rate,
            'max_depth': max_depth,
            'cv_score': avg_cv_score
        })

        print(f"n_estimators: {n_estimators}, learning_rate: {learning_rate}, max_depth: {max_depth} => CV Score: {avg_cv_score:.4f}")


        if avg_cv_score > best_score:
            best_score = avg_cv_score
            best_params = model_params

print("\nBest Hyperparameters:")
print(f"n_estimators: {best_params['n_estimators']}, learning_rate: {best_params['learning_rate']}, max_depth: {best_params['max_depth']}")
print(f"Best Cross-Validation Score: {best_score:.4f}")

start_time = tm.time()
Gradient_booster = CustomGradientBoostingClassifier(n_estimators=10, learning_rate=0.1, max_depth=6) #With best parameters found above
Gradient_booster.fit(X_train_new, y_train_new)
predictions_gb = Gradient_booster.predict(X_test_new)
end_time = tm.time()
elapsed_time = end_time - start_time
conf_matrix_gb, class_labels_gb = compute_confusion_matrix(y_test_new, predictions_gb)
print(f"Elapsed time for GBM execution: {elapsed_time:.4f} seconds")
```

```
conf_matrix_df_gb = pd.DataFrame(conf_matrix_gb, index=class_labels_gb,
columns=class_labels_gb)


plt.figure(figsize=(8, 6))

ax = sns.heatmap(conf_matrix_df_gb, annot=True, fmt="d", cmap="Blues", linewidths=0.5,
cbar=False)


plt.title("Confusion Matrix", fontsize=16)

plt.xlabel("Predicted Labels", fontsize=12)

plt.ylabel("True Labels", fontsize=12)

plt.xticks(np.arange(len(class_labels_gb)) + 0.5, class_labels_gb, rotation=45, ha="right",
fontsize=10)

plt.yticks(np.arange(len(class_labels_gb)) + 0.5, class_labels_gb, rotation=0, fontsize=10)


ax.hlines(np.arange(len(class_labels_gb) + 1), *ax.get_xlim(), colors="black", linewidth=1.5)

ax.vlines(np.arange(len(class_labels_gb) + 1), *ax.get_ylim(), colors="black", linewidth=1.5)


plt.tight_layout()

plt.show()

precision_dict, recall_dict, f1_dict = compute_precision_recall_f1(conf_matrix_gb)

precision_scores = {class_labels_gb[i]: precision_dict[i] for i in range(len(class_labels_gb))}

recall_scores = {class_labels_gb[i]: recall_dict[i] for i in range(len(class_labels_gb))}

f1_scores = {class_labels_gb[i]: f1_dict[i] for i in range(len(class_labels_gb))}


print("Precision per class for GB:", precision_scores)
```

```
print("Recall per class for GB:", recall_scores)

print("F1-Score per class for GB:", f1_scores)

n_estimators_vals = [r['n_estimators'] for r in results]

learning_rate_vals = [r['learning_rate'] for r in results]

max_depth_vals = [r['max_depth'] for r in results]

cv_scores_vals = [r['cv_score'] for r in results]


fig = plt.figure(figsize=(12, 8))

ax = fig.add_subplot(111, projection='3d')

sc = ax.scatter(n_estimators_vals, learning_rate_vals, max_depth_vals, c=cv_scores_vals,
cmap='viridis', s=100)


plt.colorbar(sc, label='Cross-Validation Score')


ax.set_xlabel('Number of Estimators (n_estimators)')

ax.set_ylabel('Learning Rate')

ax.set_zlabel('Max Depth')

ax.set_title('3D Hyperparameter Tuning of Custom Gradient Boosting Classifier')


plt.show()
```

```
data = pd.read_csv("psion_upsilon.csv")

y_df = data["class"]

y_binary = y_df.replace({"upsilon": 1, "J/psi": 0})

y_binary_counts = y_binary.value_counts()

X_df = data.drop(columns=["class","type1","type2"])  # All other columns are features

X_df = X_df.drop(columns=["eta2","px2","pz1","pz2","eta2"])

data_new = pd.concat([X_df, y_binary], axis=1)

corr_matrix = data_new.corr()

y_df = y_binary

corr_matrix#[eta2,px2,pz1,pz2,eta2]

# Extract the target variable and binary encode

y_df = data["class"]

y_binary = y_df.replace({"upsilon": 1, "J/psi": 0})


# Drop unnecessary columns and form the feature set

X_df = data.drop(columns=["class", "type1", "type2"])

X_df = X_df.drop(columns=["eta2", "px2", "pz1", "pz2", "eta2"])


# Combine the processed features with the binary target

data_new = pd.concat([X_df, y_binary], axis=1)


# Compute the correlation matrix

corr_matrix = data_new.corr()
```

```
# Extract the column names from the updated DataFrame for plotting
columns = data_new.columns

# Plotting the correlation matrix
plt.figure(figsize=(12, 10))
plt.imshow(corr_matrix, cmap='coolwarm', interpolation='none', aspect='auto')

cbar = plt.colorbar()
cbar.set_label("Correlation Coefficient", rotation=270, labelpad=20)

plt.grid(False)
plt.title("Correlation Matrix (Features and Outputs)", fontsize=14, pad=20)
plt.xticks(range(len(columns)), columns, rotation=45, ha="right", fontsize=10)
plt.yticks(range(len(columns)), columns, fontsize=10)

# Annotate the correlation values in the plot
for i in range(len(columns)):
    for j in range(len(columns)):
        value = corr_matrix.iloc[i, j]
        plt.text(j, i, f"{value:.2f}", ha="center", va="center", fontsize=8, color="black")

plt.tight_layout()
plt.show()
```

```
corr_matrix_df = pd.DataFrame(corr_matrix, index=columns, columns=columns)

corr_matrix_df

X = X_df.to_numpy()

X = X[:40000]

y = y_df.to_numpy()

y = y[:40000]


def sigmoid(z):

    return 1 / (1 + np.exp(-z))


def sigmoid_derivative(z):

    return sigmoid(z) * (1 - sigmoid(z))


def binary_cross_entropy_loss(y_true, y_pred):

    m = y_true.shape[1]

    loss = -(1/m) * np.sum(y_true * np.log(y_pred + 1e-9) + (1 - y_true) * np.log(1 - y_pred +
1e-9))

    return loss


class NeuralNetworkBinary:

    def __init__(self, layers):

        self.layers = layers

        self.params = self._initialize_weights()


    def _initialize_weights(self): #Random initialization with gaussian *0.01

        np.random.seed(42)
```

```
        params = {}
        for l in range(1, len(self.layers)):
            params[f"W{l}"] = np.random.randn(self.layers[l], self.layers[l-1]) * 0.01
            params[f"b{l}"] = np.zeros((self.layers[l], 1))
        return params
    #def _initialize_weights(self): He initialization
    #np.random.seed(42)
    #params = {}
    #for l in range(1, len(self.layers)):
        #params[f"W{l}"] = np.random.randn(self.layers[l], self.layers[l-1]) * np.sqrt(2 / self.layers[l-1])  # He Initialization
        #params[f"b{l}"] = np.zeros((self.layers[l], 1))
    #return params



    def forward_propagation(self, X):
        A = X
        cache = {"A0": A}
        for l in range(1, len(self.layers)):
            W, b = self.params[f"W{l}"], self.params[f"b{l}"]
            Z = np.dot(W, A) + b
            if l == len(self.layers) - 1:  # Output layer
                A = sigmoid(Z)  # Output layer uses sigmoid for binary classification
            else:
                A = np.maximum(0, Z)  # Hidden layers use ReLU
            cache[f"Z{l}"] = Z
```

```
        cache[f"A{l}"] = A
    return A, cache

def backward_propagation(self, X, Y, cache):
    m = X.shape[1]
    grads = {}
    L = len(self.layers) - 1
    A_final = cache[f"A{L}"]

    dZ = A_final - Y
    for l in reversed(range(1, L + 1)):
        A_prev = cache[f"A{l-1}"]
        W = self.params[f"W{l}"]
        grads[f"dW{l}"] = (1 / m) * np.dot(dZ, A_prev.T)
        grads[f"db{l}"] = (1 / m) * np.sum(dZ, axis=1, keepdims=True)
        if l > 1:  # Hidden layers
            dZ = np.dot(W.T, dZ) * sigmoid_derivative(cache[f"Z{l-1}"])
    return grads

def update_parameters(self, grads, learning_rate):
    for l in range(1, len(self.layers)):
        self.params[f"W{l}"] -= learning_rate * grads[f"dW{l}"]
        self.params[f"b{l}"] -= learning_rate * grads[f"db{l}"]

def fit(self, X, Y, epochs=1000, learning_rate=0.01, batch_size=32):
```

```
    m = X.shape[1]  # Total number of examples
    for epoch in range(epochs):
        permutation = np.random.permutation(m)  # Shuffle data
        X_shuffled = X[:, permutation]
        Y_shuffled = Y[:, permutation]

        for i in range(0, m, batch_size):
            X_batch = X_shuffled[:, i:i + batch_size]
            Y_batch = Y_shuffled[:, i:i + batch_size]

            # Forward Propagation
            A_final, cache = self.forward_propagation(X_batch)
            # Compute Loss
            loss = binary_cross_entropy_loss(Y_batch, A_final)
            # Backward Propagation
            grads = self.backward_propagation(X_batch, Y_batch, cache)
            # Update Parameters
            self.update_parameters(grads, learning_rate)

        if epoch % 10 == 0:
            print(f"Epoch {epoch}, Loss: {loss:.4f}")

def predict(self, X, threshold=0.5):
    A_final, _ = self.forward_propagation(X)
    return (A_final > threshold).astype(int)
```

```
if __name__ == "__main__":


    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

    X_train = standardize_data(X_train).T
    X_test = standardize_data(X_test).T
    y_train = y_train.reshape(1, -1)  # Reshape to match (1, m)
    y_test = y_test.reshape(1, -1)

    # Define and train the neural network
    start_time = tm.time()
    nn = NeuralNetworkBinary(layers=[15, 4, 1])  # 2 inputs -> 5 hidden -> 1 output
    nn.fit(X_train, y_train, epochs=100, learning_rate=0.01, batch_size=16)
    y_pred = nn.predict(X_test)
    accuracy = np.mean(y_pred == y_test)
    print(f"Test Accuracy: {accuracy:.2f}")

    end_time = tm.time()
    elapsed_time = end_time - start_time
    print(f"Elapsed time for NN execution: {elapsed_time:.4f} seconds")
```

```
y_test = y_test.flatten() if isinstance(y_test, np.ndarray) else y_true

y_pred = y_pred.flatten() if isinstance(y_pred, np.ndarray) else y_pred

conf_matrix, class_labels = compute_confusion_matrix(y_test, y_pred)

print("Confusion Matrix:")

print(conf_matrix)

print("Class Labels:", class_labels)

conf_matrix_df = pd.DataFrame(conf_matrix, index=class_labels, columns=class_labels)


plt.figure(figsize=(8, 6))

ax = sns.heatmap(conf_matrix_df, annot=True, fmt="d", cmap="Blues", linewidths=0.5,
cbar=False)


plt.title("Confusion Matrix", fontsize=16)

plt.xlabel("Predicted Labels", fontsize=12)

plt.ylabel("True Labels", fontsize=12)

plt.xticks(np.arange(len(class_labels)) + 0.5, class_labels, rotation=45, ha="right",
fontsize=10)

plt.yticks(np.arange(len(class_labels)) + 0.5, class_labels, rotation=0, fontsize=10)


ax.hlines(np.arange(len(class_labels) + 1), *ax.get_xlim(), colors="black", linewidth=1.5)

ax.vlines(np.arange(len(class_labels) + 1), *ax.get_ylim(), colors="black", linewidth=1.5)


plt.tight_layout()

plt.show()

precision_dict, recall_dict, f1_dict = compute_precision_recall_f1(conf_matrix)

precision_scores = {class_labels[i]: precision_dict[i] for i in range(len(class_labels))}
```

```
recall_scores = {class_labels[i]: recall_dict[i] for i in range(len(class_labels))}

f1_scores = {class_labels[i]: f1_dict[i] for i in range(len(class_labels))}


print("Precision per class:", precision_scores)

print("Recall per class:", recall_scores)

print("F1-Score per class:", f1_scores)

best_accuracy = 0

best_params = None

all_results = []


for lr in [0.001, 0.01, 0.1]:

    for hidden_units in [2,4,8,16,32]:

        for batch_size in [4, 8, 16]:

            print(f"Testing configuration: Learning Rate={lr}, Layers={hidden_units}, Batch
Size={batch_size}")

            accuracy = k_fold_cross_validation(

                X=X_train,

                y=y_train,

                k=5,

                layers=[X_train.shape[0],hidden_units, 1],

                epochs=100,

                learning_rate=lr,

                batch_size=batch_size

            )

            # Store results

            all_results.append({
```

```
            "Learning Rate": lr,
            "Layers": hidden_units,
            "Batch Size": batch_size,
            "Accuracy": accuracy
        })
        # Update best parameters if applicable
        if accuracy > best_accuracy:
            best_accuracy = accuracy
            best_params = (lr, hidden_units, batch_size)

print(f"Best Accuracy: {best_accuracy:.4f} with Parameters: "
      f"Learning Rate={best_params[0]}, Layers={best_params[1]}, Batch Size={best_params[2]}")

# Display all results
results_df = pd.DataFrame(all_results)
print(results_df)

results_df.to_csv("kfold_hyperparameter_results.csv", index=False)
fig, ax = plt.subplots(figsize=(12, 8))

# Plot each unique combination of layers and batch size manually
for hidden_units in results_df["Layers"].unique():
    subset = results_df[results_df["Layers"] == hidden_units]
    for batch_size in subset["Batch Size"].unique():
        batch_subset = subset[subset["Batch Size"] == batch_size]
```

```
        ax.plot(
            batch_subset["Learning Rate"],
            batch_subset["Accuracy"],
            marker='o',
            label=f"Hidden Units {hidden_units}, Batch Size {batch_size}"
        )

ax.set_xscale('log')  # Log scale for learning rate
ax.set_title("Accuracy vs Learning Rate for Different Hidden Units and Batch Sizes", fontsize=14)
ax.set_xlabel("Learning Rate (log scale)", fontsize=12)
ax.set_ylabel("Accuracy", fontsize=12)
ax.legend(title="Configuration", loc="upper left", bbox_to_anchor=(1.05, 1))
ax.grid(True, linestyle="--", alpha=0.6)

plt.tight_layout()

plt.show()
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

X_train = standardize_data(X_train).T
X_test = standardize_data(X_test).T
y_train = y_train.reshape(1, -1)  # Reshape to match (1, m)
y_test = y_test.reshape(1, -1)
y_train = np.asarray(y_train).flatten()
```

```
Gradient_booster = CustomGradientBoostingClassifier(n_estimators=16, learning_rate=0.01,
max_depth=6)


Gradient_booster.fit(X_train.T, y_train.T)

predictions = Gradient_booster.predict(X_test.T)


accuracy = np.mean(predictions == y_test)

accuracy

tree = DecisionTreeRegressorScratch(max_depth=4, criterion="gini")

tree.fit(X_train.T, y_train.T)


# Predictions

y_pred = tree.predict(X_test.T)

accuracy = np.mean(y_pred == y_test)

accuracy
```